



HEXLOGIC

MOBILE APPLICATION PENETRATION TESTING



“InsecureBankv2 (Android) & DVIA-v2 (iOS)”

Document Information

Project Name	Mobile Application Penetration Testing
Document Title	Mobile Application Penetration Test Report
Document Type	Final Report
Classification	Confidential & Restricted
Created By	HexLogic Offensive Security

Document History

Version	Author	Date	Change Description
1.0	HexLogic	30 May 2025	Final Report

Table of Contents

1 Introduction

This report describes the proceedings and results of the mobile application penetration test conducted by HexLogic against InsecureBankv2 (Android) & DVIA-v2 (iOS). It enlists the findings identified during the engagement, their business impact and recommended remediation.

1.1 Objective

The objective of the assessment was to evaluate the security posture of InsecureBankv2 (Android) & DVIA-v2 (iOS) and to provide a final security review report comprising a remediation strategy and recommendations to help mitigate the identified vulnerabilities and risks. The assessment was conducted presuming the malicious intent of an external adversary in order to identify associated business risk and its impact.

1.2 Assessment Duration

Test Duration	15 May 2025 – 23 May 2025
Total Days	7 Days
Note	First Test

1.3 Team Contact Details

Name	HexLogic
Contact	security@hexlogic.io
Role	Penetration Testing Team

2 Scope

This section defines the scope and boundaries of the engagement.

2.1 Target Scope

- InsecureBankv2.apk (Android test application)
- DVIA-v2.ipa (iOS test application)

2.2 Assessment Attributes

Starting Vector	Local device (static and dynamic analysis)
Target Criticality	Assessment Environment
Assessment Nature	Cautious & calculated
Assessment Conspicuity	Clear
Proof of Concept(s)	Attached wherever possible and applicable

2.3 Assessment Type and Methods

The assessment was performed in a grey-box manner, primarily manual in nature while using a limited, necessary tool-set to make the process more efficient, preserving the imitation of a real-world adversary. Testing covered static analysis and reverse engineering, local data storage, transport security, platform interaction, authentication and anti-tampering controls.

2.4 Methodology & Risk Classification

The following risk classification is used throughout this report. Findings are rated using CVSS v3.1 and business context.

Critical	Trivially exploitable issues leading to full system or data compromise; remediate immediately.
High	Serious issues allowing significant unauthorised access or impact; remediate as a priority.
Medium	Issues exploitable under certain conditions, often combined with others; schedule remediation.
Low	Limited-impact weaknesses and defence-in-depth gaps; address during routine hardening.
Info	No direct threat, but may reveal sensitive information useful to an attacker.

3 Executive Summary

The target in scope was reviewed for the adequacy of its existing security controls in accordance with the OWASP MASVS and the OWASP Mobile Application Security Testing Guide (MASTG) and industry best practices. The aim of the testing was to establish whether the target could be compromised to allow unauthorised access to sensitive data or systems.

A total of 8 vulnerabilities were identified during the assessment, comprising 0 Critical, 3 High, 4 Medium and 1 Low-risk findings. The applications store sensitive data insecurely, embed hardcoded secrets and omit certificate pinning, allowing credential theft and traffic interception on a compromised or shared device; these high-severity issues should be remediated first, followed by the platform-interaction and resilience weaknesses.

3.1 Graphical Representation of Vulnerabilities

The table below summarises the overall risk identified during the assessment.

Critical	High	Medium	Low	Total
0	3	4	1	8

4 Compliance – Detailed Summary

The final risk value of each vulnerability is derived by considering the likelihood of exploitation and the impact on the business of the assessed target.

No	Findings	Severity	CVSS 3.1	Status
1	Insecure Local Data Storage of Credentials	High	7.5	Open
2	Hardcoded Secrets in Application Package	High	7.4	Open
3	Missing Certificate Pinning — Traffic Interception	High	7.4	Open
4	Exported Components Bypass Authentication	Medium	6.5	Open
5	Root/Jailbreak Detection Bypass	Medium	6.3	Open
6	Sensitive Data Written to System Logs	Medium	5.5	Open
7	Debuggable & Backup-Enabled Application Flags	Medium	5.3	Open
8	Lack of Code Obfuscation & Anti-Tampering	Low	3.1	Open

5 Detailed Technical Summary

This section provides the technical detail for each finding, including a comprehensive description, business impact, reproduction steps, proof of concept and remediation guidance.

5.1 Insecure Local Data Storage of Credentials

High

CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

7.5
Open

Description

The application stores user credentials and session data in cleartext within SharedPreferences and a local SQLite database. On a rooted device, a shared device, or following a backup extraction, this data is readable by any other application or by an attacker with local access. Sensitive data should never be persisted unencrypted on a mobile device, where the threat model assumes the device may be lost, shared or compromised.

Affected Component

/data/data/com.android.insecurebankv2/shared_prefs/

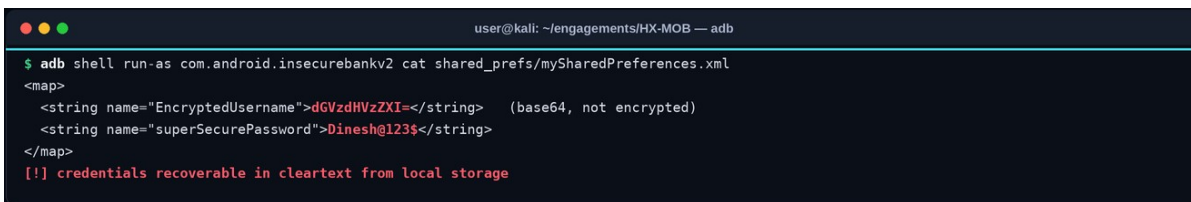
Impact

- Recovery of stored usernames, passwords and session tokens from local storage.
- Account takeover using the recovered credentials or session material.
- Exposure of sensitive personal and financial data held on the device.

Steps to Reproduce

1. Install the application and authenticate to generate stored data.
2. Use adb to pull the application's shared_prefs and database files.
3. Inspect the files and observe credentials stored in cleartext.

Proof of Concept



```

user@kali: ~/engagements/HX-MOB — adb
$ adb shell run-as com.android.insecurebankv2 cat shared_prefs/mySharedPreferences.xml
<map>
  <string name="EncryptedUsername">dGVzdHVzZXI=</string> (base64, not encrypted)
  <string name="superSecurePassword">Dinesh@123$</string>
</map>
[!] credentials recoverable in cleartext from local storage
  
```

Figure 1. Application credentials are recovered in cleartext from SharedPreferences via adb.

Recommendations

- Never store credentials on the device; use short-lived tokens and re-authenticate as needed.
- Where storage is unavoidable, use the platform keystore (Android Keystore / iOS Keychain) with hardware-backed keys.
- Disable application backups for sensitive data (android:allowBackup=false) and exclude secrets from backups.
- Encrypt local databases (e.g. SQLCipher) and validate storage handling in CI.

References

- [OWASP MASTG — Data Storage](#)

- [CWE-312](#)

5.2 Hardcoded Secrets in Application Package

High

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N

7.4
Open

Description

Decompilation of the application reveals hardcoded secrets, including an API key and a symmetric encryption key, embedded directly in the code and resources. Because mobile applications are distributed to end users, anything embedded in the package can be extracted by any user with standard reverse-engineering tools. Hardcoded secrets therefore provide no confidentiality and expose the backing services they protect.

Affected Component

[classes.dex](#) / [resources](#)

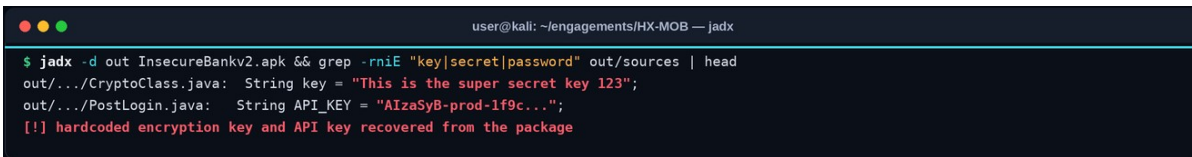
Impact

- Extraction of API keys and cryptographic keys by any user of the application.
- Abuse of backend services and decryption of data protected by the embedded key.
- Potential financial impact where keys grant access to paid or privileged APIs.

Steps to Reproduce

1. Decompile the APK with a tool such as jadx.
2. Search the decompiled sources and resources for key material and secrets.
3. Confirm the recovered key is valid against the backing service.

Proof of Concept



```

user@kali: ~/engagements/HX-MOB — jadx
$ jadx -d out InsecureBankv2.apk && grep -rniE "key|secret|password" out/sources | head
out/.../CryptoClass.java: String key = "This is the super secret key 123";
out/.../PostLogin.java: String API_KEY = "AIZA-syB-prod-1f9c...";
[!] hardcoded encryption key and API key recovered from the package

```

Figure 2. Decompiling the APK reveals a hardcoded encryption key and API key in the source.

Recommendations

- Remove secrets from the application package; never ship long-lived keys in client code.
- Obtain credentials at runtime from an authenticated backend and scope them tightly.
- Use the platform keystore for any client-side key material and rotate compromised keys immediately.
- Add secret-scanning to the build pipeline to prevent reintroduction.

References

- [OWASP MASTG — Code Quality & Secrets](#)
- [CWE-798](#)

5.3 Missing Certificate Pinning — Traffic Interception

High

CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

7.4
Open

Description

The application does not implement certificate or public-key pinning and trusts the device's user-installed certificate store. An attacker able to install a trusted certificate on the device, or positioned as a man-in-the-middle, can decrypt, inspect and modify the application's TLS traffic. Pinning is a critical control for high-assurance mobile applications because it prevents interception even when the device trust store is manipulated.

Affected Component

TLS to api backend

Impact

- Interception and modification of all application traffic, including credentials and tokens.
- Tampering with API requests and responses to manipulate application behaviour.
- Bypass of transport confidentiality on compromised or managed devices.

Steps to Reproduce

1. Configure an intercepting proxy and install its CA on the test device.
2. Route the application's traffic through the proxy.
3. Observe full plaintext request/response traffic, confirming no pinning is enforced.

Proof of Concept



```

user@kali: ~/engagements/HX-MOB — mitmproxy
$ mitmproxy --mode transparent --showhost
POST https://api.insecurebank.app/login
  username=testuser&password=Dinesh@123$
200 OK { "token": "eyJhbGciOiI..." }
[!] no certificate pinning - TLS intercepted and credentials read in plaintext
  
```

Figure 3. With a proxy CA installed, the app's TLS traffic is intercepted in plaintext — no pinning is enforced.

Recommendations

- Implement certificate or public-key pinning for the application's backend endpoints.
- Pin to a backup key as well to allow rotation without breaking clients.
- Reject user-added CAs for the application's network security configuration where appropriate.
- Combine pinning with tamper detection so pinning cannot be trivially patched out.

References

- [OWASP MASTG — Network Communication](#)
- [CWE-295](#)

5.4 Exported Components Bypass Authentication

Medium

CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N

6.5
Open

Description

The application manifest exports activities that should be internal, without permission protection. A malicious application installed on the same device, or an attacker with adb access, can invoke these activities directly via intents, reaching post-authentication screens and functionality without logging in. Exported components broaden the attack surface and can bypass the application's intended navigation and authentication flow.

Affected Component

[AndroidManifest.xml](#) — exported activities

Impact

- Direct access to post-authentication screens, bypassing the login flow.
- Invocation of sensitive functionality by other applications on the device.
- Potential disclosure or manipulation of data via crafted intents.

Steps to Reproduce

1. Inspect the manifest for activities with `android:exported=true` and no permission.
2. Use adb to start an exported, post-login activity directly.
3. Observe the protected screen render without authentication.

Proof of Concept

```

user@kali: ~/engagements/HX-MOB — adb am
$ aapt dump xmltree InsecureBankv2.apk AndroidManifest.xml | grep -A1 exported
  activity PostLogin    android:exported=true
$ adb shell am start -n com.android.insecurebankv2/.PostLogin
Starting: Intent { cmp=com.android.insecurebankv2/.PostLogin }
[!] post-login screen reached directly, bypassing authentication

```

Figure 4. An exported activity is launched directly via adb, bypassing the login flow.

Recommendations

- Set `android:exported=false` for all components that do not need to be externally reachable.
- Protect any intentionally exported components with signature-level permissions and input validation.
- Re-verify the authenticated state inside sensitive activities rather than relying on navigation.
- Review the manifest as part of the secure-build checklist.

References

- [OWASP MASTG — Platform Interaction](#)
- [CWE-926](#)

5.5 Root/Jailbreak Detection Bypass

Medium

CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:U/C:L/I:H/A:N

6.3
Open

Description

The application's root/jailbreak detection relies on simple client-side checks that can be bypassed at runtime using instrumentation frameworks such as Frida or Objection. Because the checks run on a device the attacker controls, they can be hooked and forced to return a safe result, allowing the application to run in a compromised environment where other controls (such as storage protection) are also weakened.

Affected Component

runtime integrity checks

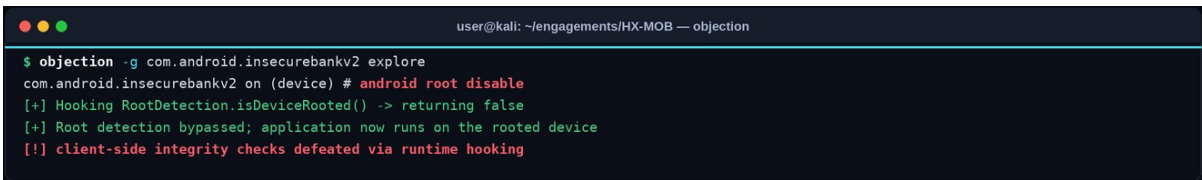
Impact

- Execution of the application on rooted/jailbroken devices despite detection logic.
- Facilitation of dynamic instrumentation, hooking and further analysis.
- Undermining of controls that assume a trusted device state.

Steps to Reproduce

1. Run the application on a rooted device and confirm detection triggers.
2. Attach Objection/Frida and apply the root-detection bypass.
3. Observe the application proceed as if on a trusted device.

Proof of Concept



```

user@kali: ~/engagements/HX-MOB — objection
$ objection -g com.android.insecurebankv2 explore
com.android.insecurebankv2 on (device) # android root disable
[+] Hooking RootDetection.isDeviceRooted() -> returning false
[+] Root detection bypassed; application now runs on the rooted device
[!] client-side integrity checks defeated via runtime hooking
  
```

Figure 5. Objection hooks the root-detection routine and forces it to report an untrusted device as safe.

Recommendations

- Treat client-side root/jailbreak detection as a defence-in-depth signal, not a security boundary.
- Use multiple, layered and obfuscated checks combined with server-side attestation (Play Integrity / DeviceCheck).
- Apply runtime application self-protection (RASP) and tamper detection to resist hooking.
- Avoid placing security-critical decisions solely on the client.

References

- [OWASP MASTG — Anti-Tampering & Resilience](#)
- [CWE-919](#)

5.6 Sensitive Data Written to System Logs

Medium

CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

5.5
Open

Description

The application writes sensitive information, including credentials and session tokens, to the system log. While modern Android restricts cross-application log access, logs remain accessible to the application itself, to users on rooted devices, during debugging, and via certain device-management tooling, creating an avoidable exposure of secrets.

Affected Component

Logcat output

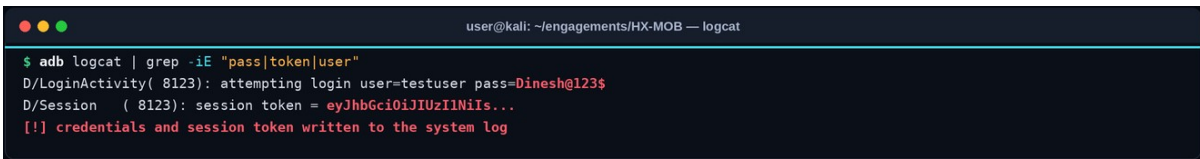
Impact

- Disclosure of credentials and tokens to anyone able to read device logs.
- Persistence of secrets in log buffers and bug reports.
- Increased blast radius when combined with other local access issues.

Steps to Reproduce

1. Run the application and perform a login.
2. Capture logcat output during the action.
3. Observe sensitive values written to the log.

Proof of Concept



```

user@kali: ~/engagements/HX-MOB — logcat
$ adb logcat | grep -iE "pass|token|user"
D/LoginActivity( 8123): attempting login user=testuser pass=Dinesh@123$
D/Session ( 8123): session token = eyJhbGciOiJIUzI1NiIs...
[!] credentials and session token written to the system log

```

Figure 6. Logcat shows the application writing credentials and a session token to the system log.

Recommendations

- Remove all logging of sensitive data and strip debug logging from release builds.
- Use a logging wrapper that redacts secrets and is disabled in production.
- Verify with static analysis that secrets are not passed to log functions.
- Scrub existing log buffers where feasible.

References

- [OWASP MASTG — Data Storage \(Logs\)](#)
- [CWE-532](#)

5.7 Debuggable & Backup-Enabled Application Flags

Medium

CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N

5.3
Open

Description

The application manifest enables `android:debuggable` and `android:allowBackup`. The `debuggable` flag allows an attacker with local access to attach a debugger and inspect or manipulate the running application, while `allowBackup` permits extraction of application data via `adb backup`, including data the application stores locally. Both flags should be disabled in production builds.

Affected Component

[AndroidManifest.xml](#)

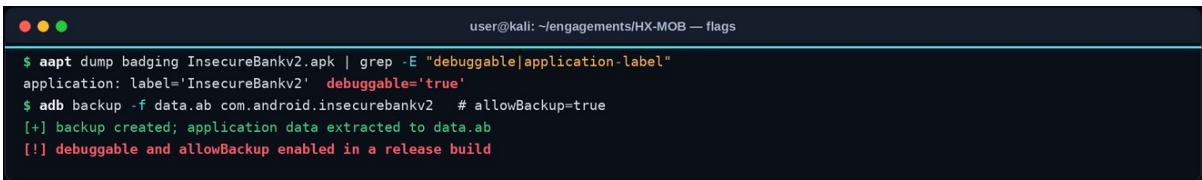
Impact

- Runtime debugging and memory inspection of the application by a local attacker.
- Extraction of application private data via the backup mechanism.
- Facilitation of further analysis and data theft.

Steps to Reproduce

1. Inspect the manifest for `android:debuggable` and `android:allowBackup`.
2. Use `adb backup` to extract application data, or attach a debugger.
3. Confirm data extraction or debugging succeeds.

Proof of Concept



```

user@kali: ~/engagements/HX-MOB — flags
$ aapt dump badging InsecureBankv2.apk | grep -E "debuggable|application-label"
application: label='InsecureBankv2' debuggable='true'
$ adb backup -f data.ab com.android.insecurebankv2 # allowBackup=true
[+] backup created; application data extracted to data.ab
[!] debuggable and allowBackup enabled in a release build

```

Figure 7. The release build ships with `debuggable` and `allowBackup` enabled, permitting debugging and data extraction.

Recommendations

- Set `android:debuggable=false` and `android:allowBackup=false` (or define a strict backup rule set) for release builds.
- Enforce secure manifest settings through the release build configuration.
- Verify flags automatically during CI to prevent regressions.

References

- [OWASP MASTG — Platform & Build Settings](#)
- [CWE-489](#)

5.8 Lack of Code Obfuscation & Anti-Tampering

Low

CVSS:3.1/AV:L/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N

3.1
Open

Description

The application is shipped without code obfuscation or integrity verification, so its logic, class and method names and string constants are readily readable after decompilation. While obscurity is not a security control in itself, the complete absence of obfuscation and tamper detection lowers the cost of reverse engineering, secret discovery and the creation of repackaged, trojanised versions of the application.

Affected Component

`classes.dex`

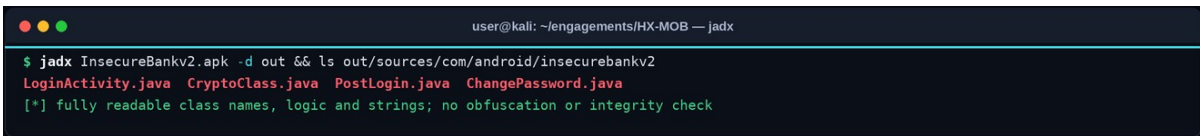
Impact

- Reduced effort for attackers to understand logic and locate secrets and controls.
- Easier creation of repackaged or trojanised builds for distribution.
- Facilitation of the other findings in this report.

Steps to Reproduce

1. Decompile the application and review the clarity of the output.
2. Note meaningful class/method names and readable string constants.
3. Confirm no integrity self-check is present.

Proof of Concept



```

user@kali: ~/engagements/HX-MOB — jadx
$ jadx InsecureBankv2.apk -d out && ls out/sources/com/android/insecurebankv2
LoginActivity.java  CryptoClass.java  PostLogin.java  ChangePassword.java
[*] fully readable class names, logic and strings; no obfuscation or integrity check

```

Figure 8. Decompiled output shows clear class names and logic with no obfuscation or tamper detection.

Recommendations

- Apply code obfuscation (e.g. R8/ProGuard, or commercial tooling) to release builds.
- Add integrity and tamper-detection checks combined with server-side attestation.
- Strip debug symbols and unnecessary metadata from release artefacts.

References

- [OWASP MASTG — Resilience](#)
- [CWE-656](#)

THANK YOU

